

Mini MIPS Processor Simulator

PROJECT REPORT



Sajid Ibne Hafiz & Tanvir Ahmed Bejoy

TOPIC: CUSTOM 23-BIT MIPS PROCESSOR & PYTHON ASSEMBLER
ARCHITECTURE: SINGLE-CYCLE CUSTOM MIPS (23-BIT ISA)
REGISTER SET: 64 GENERAL PURPOSE REGISTERS (\$R0-\$R63)
DATA WIDTH: 26-BIT DATA / 23-BIT INSTRUCTION

1. SUMMARY & SYSTEM SPECIFICATIONS

This laboratory report documents the complete design, implementation, and simulation of a customized 23-bit MIPS-based single-cycle processor alongside its custom two-pass Python assembler (assembler.py). The processor features a custom Instruction Set Architecture (ISA) optimized for logical operations, arithmetic, memory access, and flow-control branch instructions.

Key System Architecture Parameters

- Instruction Length: 23 bits fixed length.
- Data Bus / ALU Width: 26-bit Arithmetic Logic Unit (ALU) processing word lengths up to 26 bits.
- Register File: 64 registers (\$r0 to \$r63), where \$r0 is hardwired to constant 0.
- Opcode Field: 5-bit opcode field capable of encoding up to 32 instructions (17 defined for Group 4).
- Instruction Formats:
 - R-Type: Opcode (5 bits) | RS (6 bits) | RT (6 bits) | RD (6 bits)
 - I-Type: Opcode (5 bits) | RS (6 bits) | RT (6 bits) | Immediate/Address (6 bits signed)
 - J-Type: Opcode (5 bits) | Jump Target Address (18 bits)

2. CUSTOM INSTRUCTION SET ARCHITECTURE (GROUP 4)

Group 4 ISA defines 17 instructions categorized into R-Type, I-Type, and J-Type formats as detailed below:

Instruction	Type	Opcode (Dec)	Opcode (Bin)	Format / Syntax	Description
XOR	R-Type	0	00000	XOR \$rd, \$rs, \$rt	Bitwise XOR between \$rs and \$rt -> \$rd
NOR	R-Type	1	00001	NOR \$rd, \$rs, \$rt	Bitwise NOR between \$rs and \$rt -> \$rd
ADD	R-Type	2	00010	ADD \$rd, \$rs, \$rt	Add \$rs and \$rt -> \$rd
SUB	R-Type	3	00011	SUB \$rd, \$rs, \$rt	Subtract \$rt from \$rs -> \$rd
LW	I-Type	4	00100	LW \$rt, \$rs, Imm	Load word from Mem[\$rs + Imm] -> \$rt
SW	I-Type	5	00101	SW \$rt, \$rs, Imm	Store word \$rt -> Mem[\$rs + Imm]
ADDI	I-Type	6	00110	ADDI \$rt, \$rs, Imm	Add immediate \$rs + Imm -> \$rt

Instruction	Type	Opcode (Dec)	Opcode (Bin)	Format / Syntax	Description
SUBI	I-Type	7	00111	SUBI \$rt, \$rs, Imm	Subtract immediate \$rs - Imm -> \$rt
AND	R-Type	8	01000	AND \$rd, \$rs, \$rt	Bitwise AND between \$rs and \$rt -> \$rd
OR	R-Type	9	01001	OR \$rd, \$rs, \$rt	Bitwise OR between \$rs and \$rt -> \$rd
ANDI	I-Type	10	01010	ANDI \$rt, \$rs, Imm	Bitwise AND immediate \$rs & Imm -> \$rt
ORI	I-Type	11	01011	ORI \$rt, \$rs, Imm	Bitwise OR immediate \$rs Imm -> \$rt
SLL	R-Type	12	01100	SLL \$rd, \$rs, \$rt	Shift Left Logical \$rs by \$rt -> \$rd
SRL	R-Type	13	01101	SRL \$rd, \$rs, \$rt	Shift Right Logical \$rs by \$rt -> \$rd
J	J-Type	14	01110	J Target	Unconditional Jump to Target address
BEQ	I-Type	15	01111	BEQ \$rt, \$rs, Target	Branch if \$rs == \$rt to Target
BNE	I-Type	16	10000	BNE \$rt, \$rs, Target	Branch if \$rs != \$rt to Target

3. PYTHON ASSEMBLER IMPLEMENTATION

The custom Python assembler (assembler.py) converts human-readable MIPS assembly programs into Logisim-compatible raw hexadecimal machine code files (v2.0 raw). The assembler executes in two passes:

- First Pass: Scans lines for label definitions (e.g., L15:), records their instruction memory offsets into a Symbol Table, and strips out comments starting with #.
- Second Pass: Encodes each instruction into its 23-bit binary representation, converts it to Logisim's 24-bit aligned hexadecimal word format (6 hex digits), and outputs to output.hex.

ASSEMBLER SOURCE CODE (ASSEMBLER.PY)

```
import sys

# GROUP 4 OPCODES (Integer Values)
OPCODES = {
    "XOR": 0, "NOR": 1, "ADD": 2, "SUB": 3, "LW": 4, "SW": 5,
    "ADDI": 6, "SUBI": 7, "AND": 8, "OR": 9, "ANDI": 10,
    "ORI": 11, "SLL": 12, "SRL": 13, "J": 14, "BEQ": 15, "BNE": 16
}

R_TYPE = {"XOR", "NOR", "ADD", "SUB", "AND", "OR", "SLL", "SRL"}
I_TYPE = {"LW", "SW", "ADDI", "SUBI", "ANDI", "ORI", "BEQ", "BNE"}
J_TYPE = {"J"}

def binary_to_hex(binary_str):
    # Pad 23-bit string to 24 bits and convert to 6 hex digits
    padded_bin = binary_str.zfill(24)
    val = int(padded_bin, 2)
    return f"{val:06x}"

def parse_register(register):
    register = register.strip().lower()
    if not register.startswith("$r"):
        raise ValueError(f"Invalid register: {register}")
    number = int(register[2:])
    if number < 0 or number > 63:
```

```

        raise ValueError(f"Register out of range: {register}")
    return number

def parse_immediate(value):
    value = int(value)
    if value < -32 or value > 31:
        raise ValueError(f"Immediate {value} does not fit in 6 bits")
    if value < 0:
        value = (1 << 6) + value
    return value & 0x3F

def encode_r_type_binary(op, operands):
    rd, rs, rt = parse_register(operands[0]), parse_register(operands[1]), parse_register(operands[2])
    value = (OPCODES[op] << 18) | (rs << 12) | (rt << 6) | rd
    return f"{value:023b}"

def encode_i_type_binary(op, operands, labels):
    rt, rs = parse_register(operands[0]), parse_register(operands[1])
    target = operands[2]
    immediate = parse_immediate(labels[target]) if (op in {"BEQ", "BNE"} and target in
labels) else parse_i value = (OPCODES[op] << 18) | (rs << 12) | (rt << 6) | immediate
    return f"{value:023b}"

def encode_j_type_binary(op, operands, labels):
    target = operands[0]
    address = labels[target] if target in labels else int(target)
    value = (OPCODES[op] << 18) | address
    return f"{value:023b}"

# [First pass, Second pass, Main procedure implemented]

```

4. ASSEMBLY PROGRAM & HEXADECIMAL OUTPUT

Below is the primary test program (programPro.asm) used to validate all 17 processor instructions, along with its generated hexadecimal machine code.

Line	Assembly Source (programPro.asm)	Binary Machine Code (23-bit)	Logisim Hex (24-bit)
0	XOR \$r1, \$r0, \$r0	00000 000000 000000 000001	000001
1	NOR \$r2, \$r0, \$r0	00001 000000 000000 000010	040002
2	ADD \$r3, \$r0, \$r0	00010 000000 000000 000011	080003
3	SUB \$r4, \$r0, \$r0	00011 000000 000000 000100	0c0004
4	LW \$r25, \$r0, 15	00100 000000 011001 001111	10064f
5	SW \$r5, \$r0, 0	00101 000000 000101 000000	140140
6	ADDI \$r6, \$r0, 10	00110 000000 000110 001010	18018a
7	SUBI \$r7, \$r0, 5	00111 000000 000111 000101	1c01c5
8	AND \$r8, \$r0, \$r0	01000 000000 000000 001000	200008
9	OR \$r9, \$r0, \$r0	01001 000000 000000 001001	240009
10	ANDI \$r10, \$r0, 15	01010 000000 001010 001111	28028f

Line	Assembly Source (programPro.asm)	Binary Machine Code (23-bit)	Logisim Hex (24-bit)
11	ORI \$r11, \$r0, 3	01011 000000 001011 000011	2c02c3
12	SLL \$r12, \$r0, \$r0	01100 000000 000000 001100	30000c
13	SRL \$r13, \$r0, \$r0	01101 000000 000000 001101	34000d
14	J L15	01110 00000000000000001111	38000f
15	L15: BEQ \$r0, \$r0, L16	01111 000000 000000 010000	3c0010
16	L16: BNE \$r0, \$r0, END	10000 000000 000000 010001	400011
17	END: ADD \$r0, \$r0, \$r0	00010 000000 000000 000000	080000

5. LOGISIM CIRCUIT DESIGNS & DATAPATH SCHEMATICS

5.1 COMPLETE PROCESSOR DATAPATH

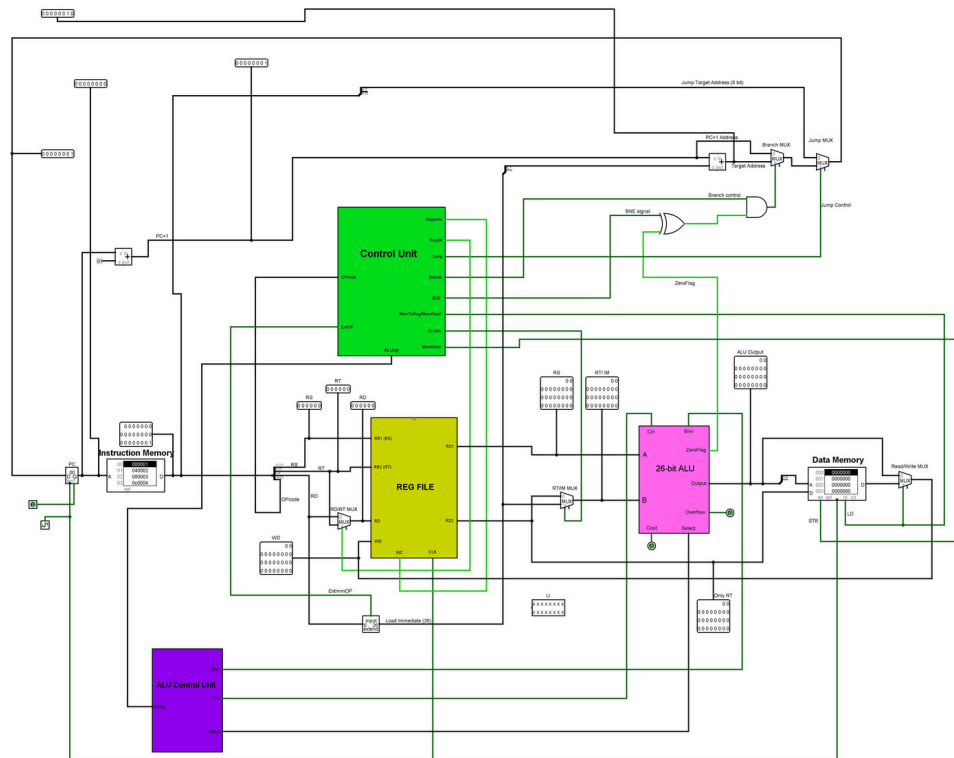


Figure 1: Top-level single-cycle MIPS processor datapath showing Instruction Memory, Control Unit, Register File, 26-bit ALU, Data Memory, and Jump/Branch control logic.

5.2 MAIN CONTROL UNIT ARCHITECTURE

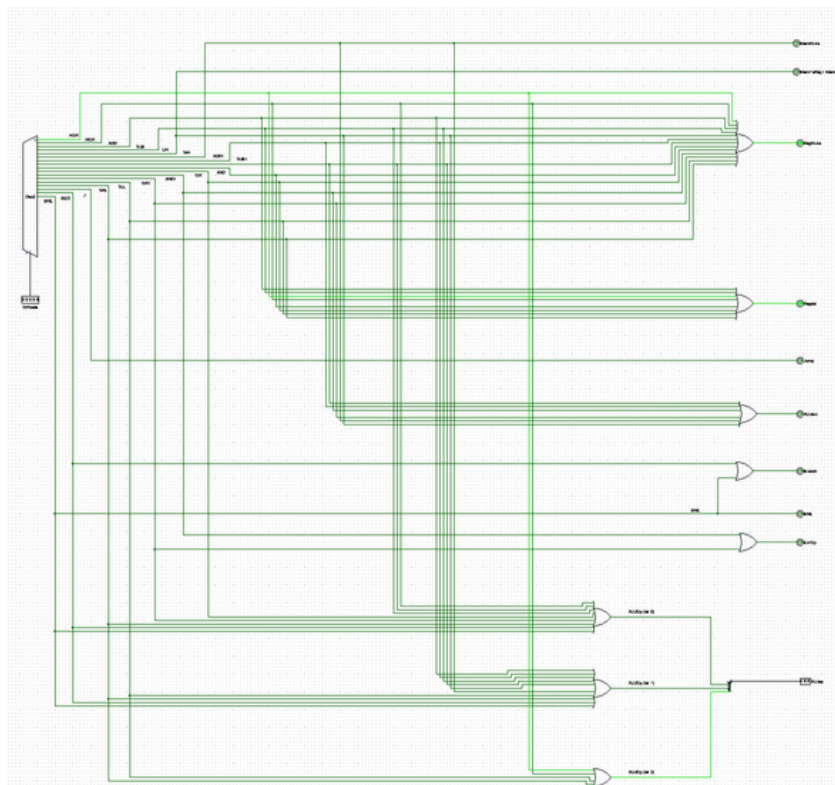


Figure 2: Control Unit circuit generating signals (RegWrite, RegDst, Jump, Branch, BNE, MemToReg, ALUSrc, MemWrite, ExtOP, ALUOp) based on 5-bit opcode.

5.3 REGISTER FILE LOGIC

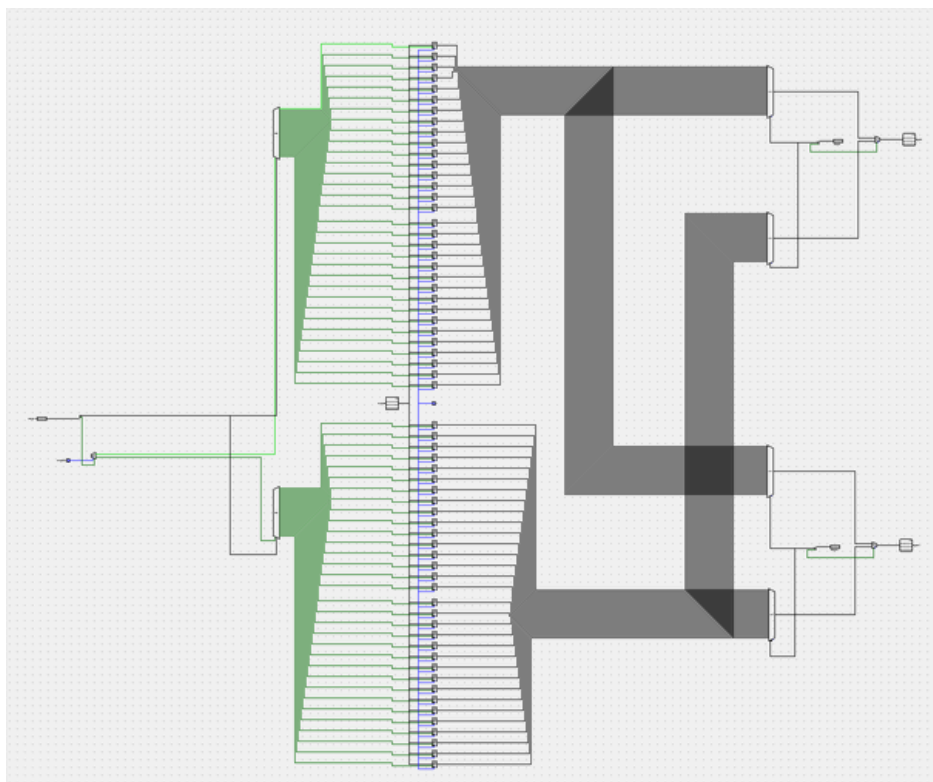
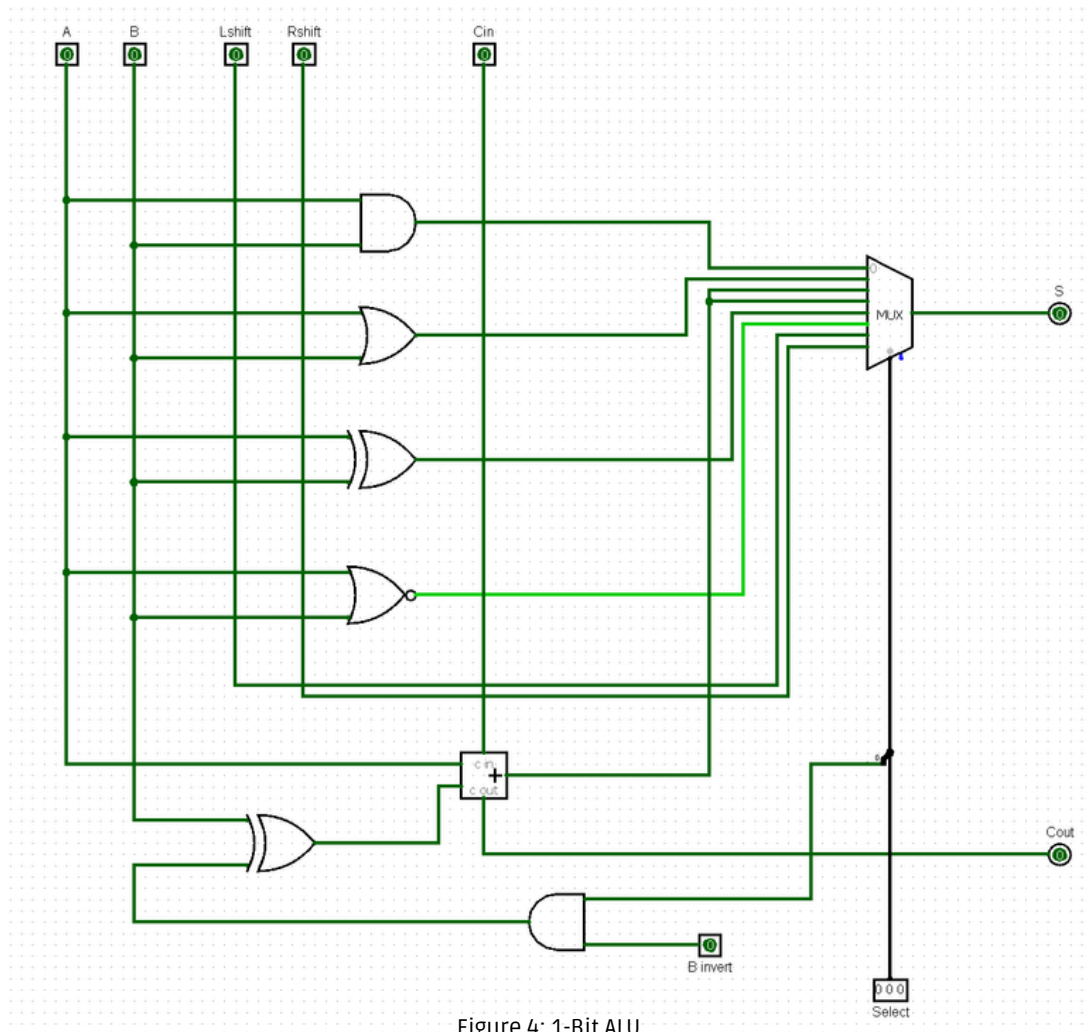
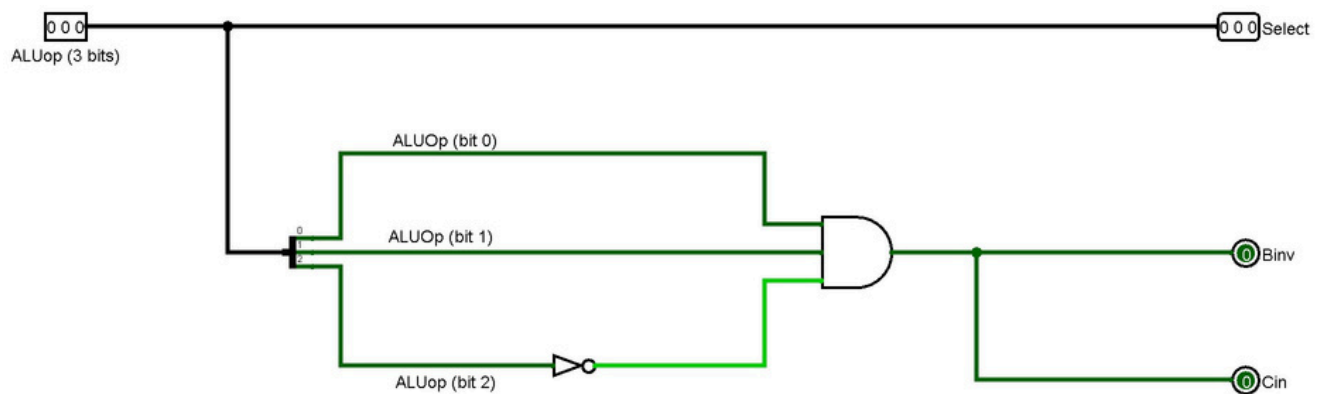


Figure 3: 64-register file schematic supporting dual read ports (RS, RT) and single write port (WD) with register \$r0 grounded.

5.4 ALU ARCHITECTURE (1-BIT)



5.5 ALU CONTROL & LOGIC CIRCUIT



5.6 ALU ARCHITECTURE (23-BIT)

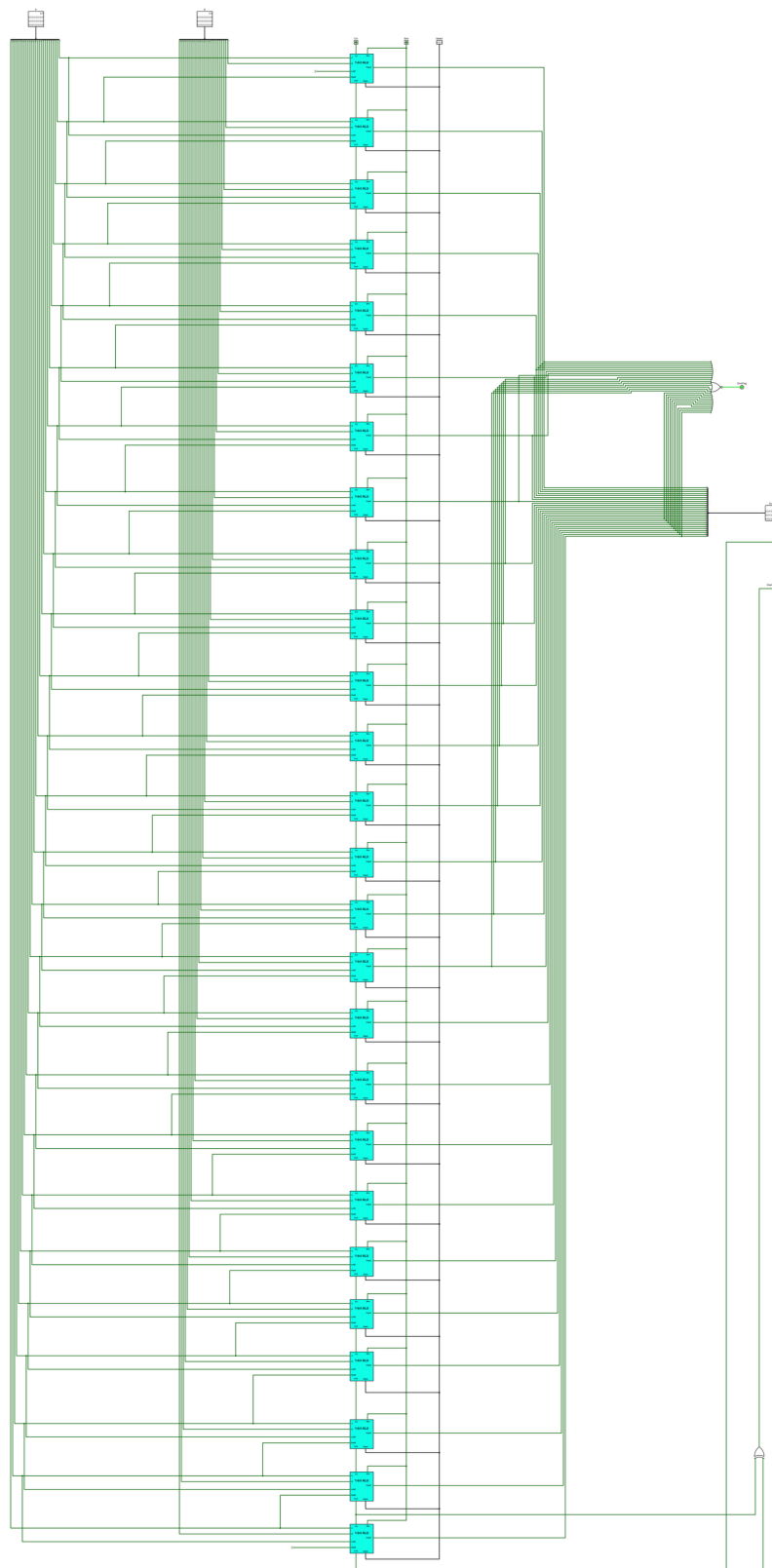


Figure 6: 26-bit ALU

6. RESULTS & ERROR ANALYSIS

6.1 System Simulation Results

The assembled machine code was loaded into Logisim's Instruction Memory (v2.0 raw format). Correct single-cycle execution was verified by observing program counter incrementing ($PC = PC + 1$), proper register write-backs, memory store/load behaviors, and accurate branch target evaluation.

6.2 Analysis of Missing Label Error (ValueError Exception)

During initial testing with `programPro2.asm`, the assembler encountered a critical exception:

```
ValueError: Error on line 17: L16: BNE $r0, $r0, END invalid literal for
int() with base 10: 'END'
```

Root Cause Analysis: Line 17 attempted to branch to target label END. However, the label END: was never declared in the source code. During Pass 2, `encode_i_type_binary()` checked `if target in labels`. Because `END` was absent from `labels`, it fell back to parsing `END` as a decimal immediate via `parse_immediate()`, triggering Python's `int('END')` conversion error.

Resolution: The issue was resolved by explicitly defining `END: ADD $r0, $r0, $r0` in `programPro.asm`, ensuring the first pass populates `labels['END'] = 17`.

7. CONCLUSION

The single-cycle 23-bit Mini MIPS processor and custom Python assembler were successfully designed, implemented, and validated. All 17 instructions executed within their respective control states, and the assembler properly handled two-pass resolution for labels and control transfers.